嵌入式 Linux 系统

malloc 函数的前世今生

一块内存的旅程

动态内存分配的背景、机制与用法

何伟明

24 电信 4 班　202434310409

2026.10.22



作业导览

课程讲解 · 原理动画 · 交互创意案例

01 课程讲解

背景与发展 · 分配机制 · 函数用法与注意事项

本文件第 3—20 页

02 malloc 原理动画

中文旁白与中文字幕 · 约 2 分 36 秒

https://mwhmwhyyds-eng.github.io/malloc-memory-lab/watch.html

03 内存旅程实验室

交互探索申请、初始化、读取与释放 · 附 C 源码

https://mwhmwhyyds-eng.github.io/malloc-memory-lab/lab.html

https://github.com/mwhmwhyyds-eng/malloc-memory-lab

动态内存分配解决什么问题

数据规模可能在运行时才确定

普通定长数组

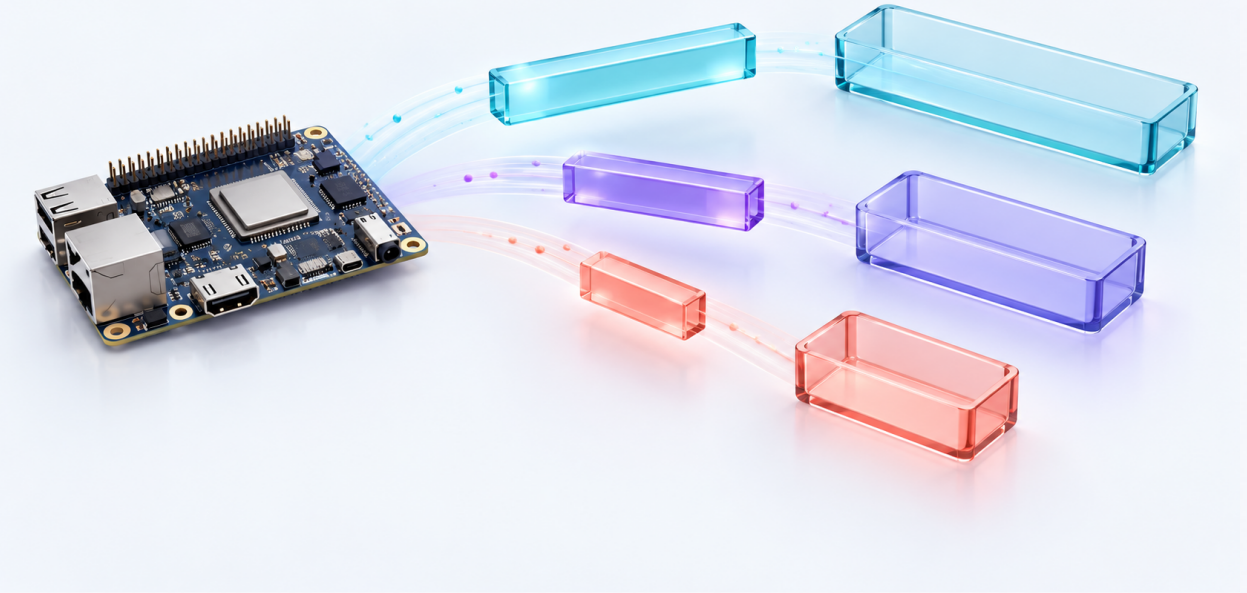
```
int a[100];
```

这一声明的元素数量固定为 100 。
输入变大可能不够用，较小时又有空余。

动态申请

```
int *p = malloc(n * sizeof *p);
```

根据运行时的 n 确定申请规模。
仍需检查大小与失败，并在使用后释放。



不同长度的数据，需要不同大小的缓冲区

AI 场景示意，非实际硬件或分配器布局

接口标准化与分配器实现

接口规定怎样调用，分配器决定内部怎样管理内存。

标准接口

C89 → `malloc / free`

规定参数、返回值与使用约定

实现演进

`dlmalloc` → `ptmalloc` → `glibc`

1987 年开始编写

源自 dlmalloc

源自 ptmalloc

1987 是 dlmalloc 的编写起点。下文内部机制以 glibc 为例。

应用、分配器与 Linux 内核

应用程序

```
malloc(size)
free(p)
```

调用库接口 →

C 库分配器

管理内存块
选择可复用空间

需要扩展时 →

Linux 内核

```
brk / mmap
munmap
```

已有空间可用

分配器直接复用内存块，
这次 malloc 可能无需系统调用。

需要更多空间

分配器可借助内核扩展
进程虚拟地址空间。

malloc 的接口与返回值

```
#include <stdlib.h>

void *malloc(size_t size);
```

size 所需字节数，类型为 size_t 。

返回值 成功时返回指针，失败时返回 NULL 。

malloc 不会初始化所分配的内存，读取前应先写入有效数据。

申请之后

检查是否成功

先初始化，再读取

最后调用 free

一次正确的申请与释放

```c
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int *p = malloc(4 * sizeof *p);
    if (p == NULL) return EXIT_FAILURE;
    for (int i = 0; i < 4; ++i)
        p[i] = i + 1;
    printf("%d\n", p[3]);
    free(p);
    return EXIT_SUCCESS;
}
```

01 申请 4 个 int 的空间

02 失败时停止使用指针

03 初始化后再读取 p[3]

04 使用结束后释放

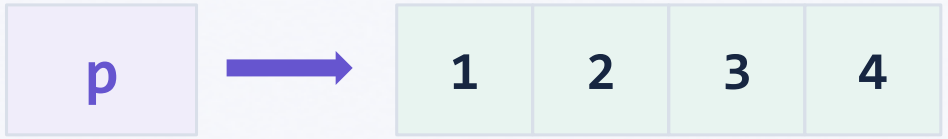
C 语言中，malloc 返回的 void * 可直接赋给对象指针，无需强制类型转换。

指针变量与内存对象的生命周期

```c
int *make_values(void) {
    int *p = malloc(4 * sizeof *p);
    if (p != NULL)
        for (int i = 0; i < 4; ++i)
            p[i] = i + 1;
    return p;
}
```

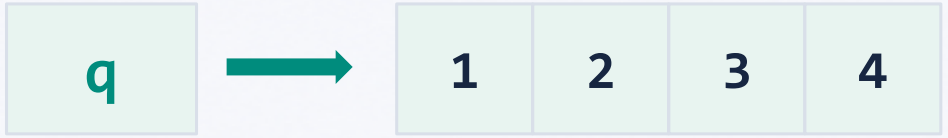
```c
int *q = make_values();
if (q != NULL) free(q);
```

函数返回前



同一分配对象

函数返回后



局部 p 已结束，q 接续持有指针值。
分配对象持续存在，直到 free(q) 。

分配器如何管理内存块

概念示意，不代表 glibc 字段排列，也不按真实比例绘制。

chunk （内存块）　　**p： 返回指针**



管理信息	应用可访问区域	内部开销
大小、状态等	从 p 开始，请求的 size 字节	依实现与请求而异

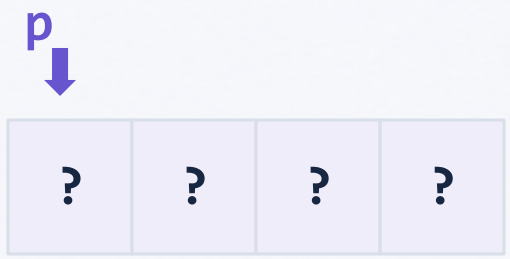
访问边界　　应用按请求大小访问，不能越界读写管理信息或其他空间。

arena （管理区域）组织多个 chunk，已释放的块可能保留以供复用。

一块内存的旅程

01 申请

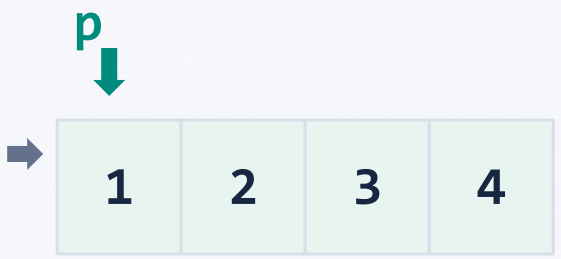
```
malloc(4 * sizeof *p)
```



成功后获得空间
内容尚未初始化

02 写入

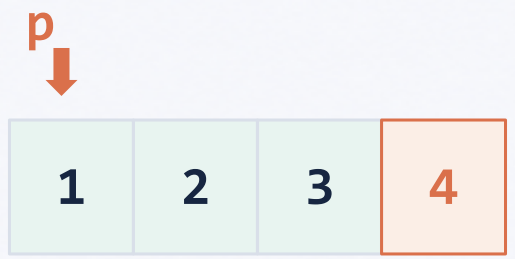
```
p[i] = i + 1
```



逐个写入有效值
此时可读取元素

03 读取

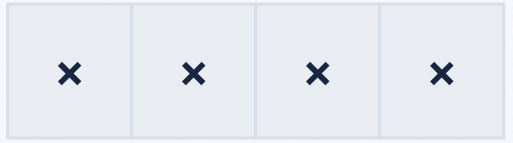
```
p[3]
```



读取第 4 个元素
输出结果为 4

04 释放

```
free(p)
```

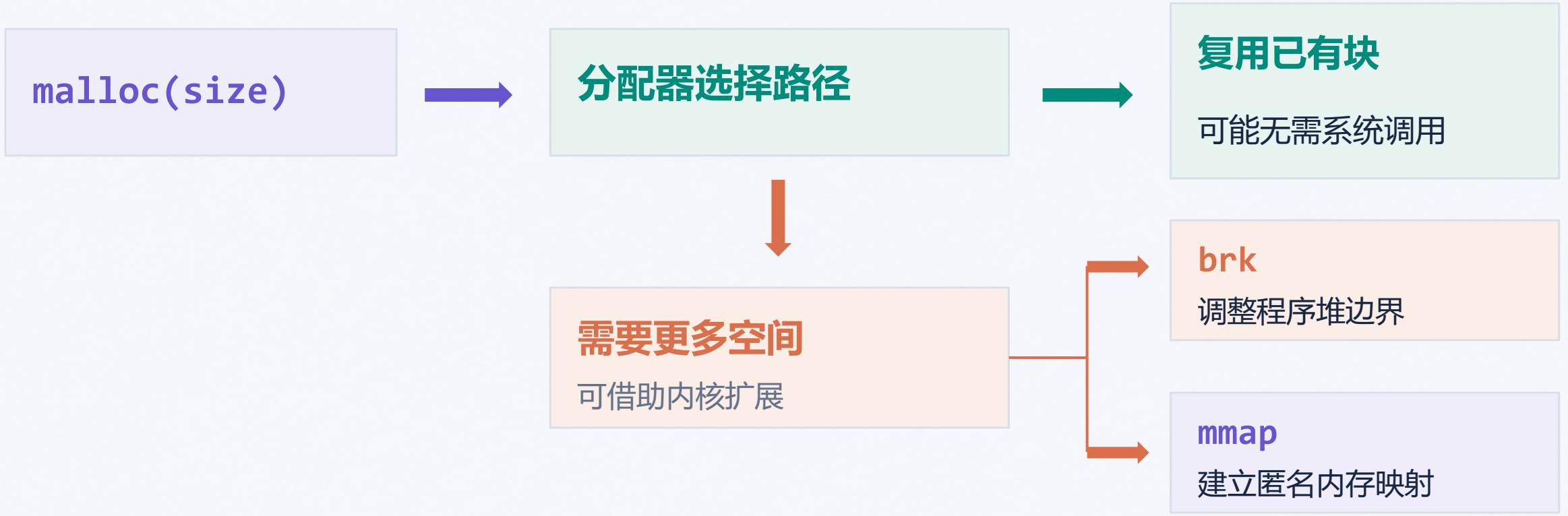


原对象使用结束
应用不能再访问

释放图中的灰色表示访问失效，不表示内容被清零。

glibc 如何获得与扩展内存

简化说明：选择取决于请求大小、现有空间和配置。



mmap 阈值可动态调整。 Linux 下返回非空也不保证后续所有物理内存都可用。

free 之后发生了什么

应用层面　对象的使用权结束。即使旧数据看似仍在，也不能继续读取或写入。

分配器层面　空间可能进入缓存或其他可用结构，等待后续申请，也可能被归还。

操作系统层面　是否归还内存取决于分配器策略，进程内存指标不一定立即下降。

free 结束对象的使用，何时归还操作系统由分配器策略决定。

内存碎片与连续空间

教学模型：固定区域内申请连续空间，不扩展，也不另开映射。

空闲 8 单位	占用 8 单位	空闲 8 单位	占用 8 单位

新请求：连续 12 单位

12 单位

总空闲量 16

最大连续空闲段 8

两个空闲段被占用块隔开，当前区域容不下这一连续请求。

外部碎片来自空闲区域分散，内部浪费可来自块内取整或对齐。

malloc、calloc、realloc 与 free

函数	主要作用	数据状态	注意事项
malloc	申请指定字节数	不初始化内容	先检查返回值
calloc	申请元素数组	按字节清零	检查分配失败 能检测乘法溢出
realloc	调整已有块大小	保留新旧大小较小范围内的数据	新增部分不初始化 可能移动对象
free	释放分配得到的对象	原对象不再可用	不能重复释放 free(NULL) 不做操作

可移植代码应避免依赖 malloc(0) 或 realloc(p, 0) 的特殊行为。

realloc 的安全写法

前提： p 有效， new_n > 0 ，并已检查大小计算不会溢出。

```
int *tmp = realloc(p, new_n * sizeof *p);
if (tmp == NULL) {
    free(p);
    return EXIT_FAILURE;
}
p = tmp;
```

失败时

原块仍有效。示例选择释放它并退出，也可按程序策略保留继续使用。

成功时

使用新返回值。旧的别名指针需要重新建立。

用临时指针接收结果，避免失败时覆盖唯一的旧指针而造成泄漏。

大小计算与边界检查

```c
#include <stdint.h>
#include <stdlib.h>
int *alloc_ints(size_t n) {
    int *p;
    if (n == 0 || n > SIZE_MAX / sizeof *p)
        return NULL;
    return malloc(n * sizeof *p);
}
```

乘法溢出

先用除法比较上界，再计算元素数乘以元素大小。

字符串终止符

字符串除内容外，还需为结尾的 \0 额外预留 1 字节。同时检查加法溢出。

sizeof p 是指针大小， sizeof *p 才对应它指向的元素类型。

常见错误：泄漏、越界与未初始化读取

错误	问题是什么	改进方式
内存泄漏	最后一个可用引用丢失前未释放 长时间重复发生会积累占用	明确谁负责释放 同时覆盖错误退出路径
访问越界	4 个元素的合法下标为 0 至 3 访问 p[4] 超出范围	把元素数量与指针一起传递 在使用处检查边界
未初始化读取	malloc 只提供空间 不能假定内容已经为 0	先写入有效值 需要按字节清零时可选 calloc

越界和未初始化读取可能导致未定义行为，不能依靠某次运行结果判断正确性。

常见错误：释放后使用与重复释放

假设 p 是成功分配后尚未释放的有效指针。

错误片段：释放后继续访问

```
free(p);
printf("%d\n", p[0]);
```

错误片段：同一对象释放两次

```
free(p);
free(p);
```

两种操作都属于未定义行为，可能表现为崩溃、数据损坏或暂时没有明显异常。

p = NULL 只改变变量 p，其他指向同一对象的别名指针仍需处理。

嵌入式 Linux 中的使用取舍

使用情形	设计时可考虑什么
启动时规模已知	集中分配并检查失败，避免运行中反复申请同类对象
数据长度随输入变化	动态申请或扩容，并限制输入大小与总内存预算
关键路径要求可预测时延	优先评估预分配或有明确时延保证的方案
长期运行服务	明确对象所有权，检查错误路径中的释放，关注峰值占用

线程安全只说明并发调用的安全属性，不能据此认定调用耗时恒定。

malloc 的背景、机制与用法

背景 运行中的规模变化与生命周期需求，使动态分配成为必要工具。

机制 分配器管理和复用内存块，需要时借助内核扩展地址空间。

用法 计算大小、检查失败、初始化后使用，并在使用结束后释放。

管理好一块内存，既要理解接口语义，也要明确每个对象由谁负责。

谢谢观看

原理视频

https://mwhmwhyyds-eng.github.io/malloc-memory-lab/watch.html

交互案例

https://mwhmwhyyds-eng.github.io/malloc-memory-lab/lab.html

GitHub 项目与源码

https://github.com/mwhmwhyyds-eng/malloc-memory-lab

